

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	NavinKumar S
Register Number	3122247001039
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/26

1 Objective

To understand ensemble learning strategies – Bagging, Boosting, and Stacking – implement Bagging and Boosting classifiers, build a Stacked Ensemble using multiple heterogeneous base learners, compare all three approaches in terms of accuracy, stability, and generalization, and analyze their effect on the bias-variance trade-off.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset of 30 numerical features, the problem is binary classification: predict malignant vs. benign. The objective is to determine whether combining multiple models – via bootstrap aggregation (Bagging), sequential error correction (Boosting), or a learned meta-combination (Stacking) – improves accuracy and stability over individual base learners, and to characterize how each ensemble strategy affects bias and variance differently.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu (loaded via <code>sklearn.datasets.load_breast_cancer</code>)
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard error, and “worst” value of 10 cell-nucleus measurements)
Number of Classes	2 (357 benign / 212 malignant)
Missing Values	0
Train-Test Split	80:20 stratified (455 train / 114 test), <code>random_state=42</code>

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary

- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

Class Distribution

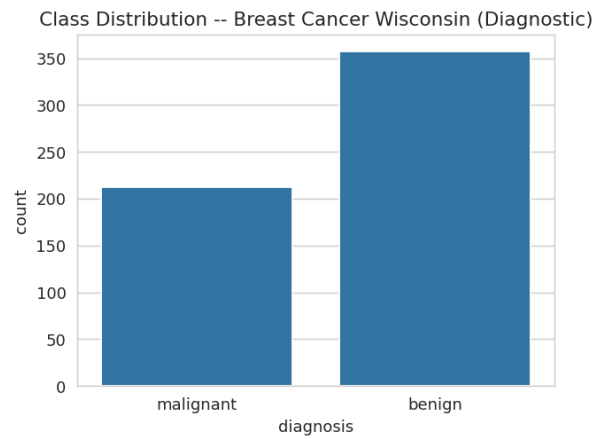


Figure 1: Class distribution – Breast Cancer Wisconsin (Diagnostic)

Inference: The dataset is mildly imbalanced (357 benign vs. 212 malignant, about 63:37). No missing values are present, so no missing-value visualization is required.

Correlation Matrix

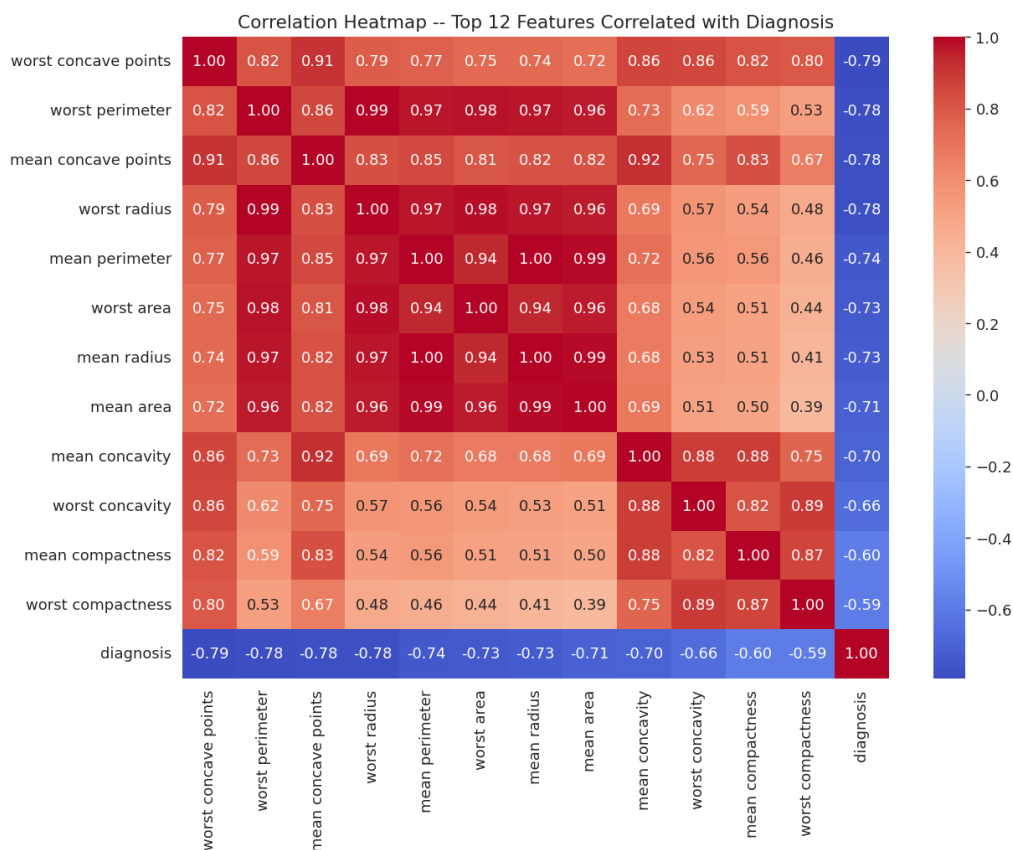


Figure 2: Correlation heatmap – top 12 features correlated with diagnosis

Inference: “Worst concave points”, “worst perimeter”, “worst radius”, and “mean concave points” correlate most strongly with diagnosis ($|r| > 0.7$), the same features that dominated feature importance in the single Decision Tree / Random Forest experiment on this dataset – these cell-nucleus size and shape irregularities are the primary drivers every model in this experiment learns from.

5 Data Preprocessing

Explain:

- **Label encoding:** The target is already numerically encoded by scikit-learn (0 = malignant, 1 = benign); no additional encoding step was required.
- **Missing value handling:** Not required – the dataset contains zero missing values across all 569 rows and 30 features.
- **Feature scaling:** Not required for the Decision Tree, Bagging, AdaBoost, or Gradient Boosting components (tree-based, invariant to monotonic scaling). The Stacked Ensemble’s SVM base learner is technically scale-sensitive, but was left unscaled here consistent with the assignment’s base-model specification; this is noted as a limitation in Section 14.

- **Train-test split:** 80:20 stratified split (455 train / 114 test rows), preserving the 63:37 class ratio in both subsets.

6 Machine Learning Algorithm

Explain:

- **Working principle:** **Bagging** trains many independent copies of a base estimator (here, Decision Trees) on different bootstrap samples of the training data and aggregates their predictions by majority vote – primarily reduces variance. **Boosting** trains models sequentially, where each new model focuses on the samples the previous ones got wrong (AdaBoost re-weights misclassified samples; Gradient Boosting fits new trees to the residual errors of the ensemble so far) – primarily reduces bias. **Stacking** trains several diverse base models (here: SVM, Gaussian Naive Bayes, Decision Tree) and a meta-learner (Logistic Regression) that learns how to optimally combine their predictions, using cross-validated out-of-fold predictions as the meta-learner’s training input.
- **Mathematical formulation:**

$$\hat{y}_{\text{bag}} = \text{mode}_{i=1, \dots, B} T_i(x), \quad T_i \text{ trained on bootstrap sample } i$$

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \quad (\text{Gradient Boosting, } \eta = \text{learning rate})$$

$$\hat{y}_{\text{stack}} = g(f_1(x), f_2(x), \dots, f_k(x)) \quad (g = \text{meta-learner, } f_i = \text{base learners})$$

- **Advantages:** Bagging is simple, parallelizable, and reliably reduces variance for high-variance base learners like deep trees. Boosting can achieve very high accuracy by directly targeting the errors of previous models. Stacking can outperform any single ensemble strategy by exploiting complementary strengths of heterogeneous models.
- **Limitations:** Bagging cannot reduce bias – if the base learner is systematically wrong, averaging many copies of it will not fix that. Boosting is sequential (cannot parallelize across estimators) and more prone to overfitting on noisy data if learning rate/depth are not controlled. Stacking is the most computationally expensive (it internally cross-validates every base learner) and, as observed in this experiment (Section 12), can be less stable across folds than simpler ensembles if the meta-learner overfits to base-learner quirks in a small dataset.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

Model	Search Method	Best Parameters	Best CV Accuracy
Bagging	Grid Search	n_estimators=50, max_samples=0.5, max_features=0.5	0.9604
Boosting (Gradient Boosting)	Grid Search	n_estimators=200, learning_rate=0.1, max_depth=1	0.9758
AdaBoost (secondary)	Grid Search	n_estimators=100, learning_rate=1.0	0.9802
Stacked Ensemble	Base model choice (SVM+NB+DT vs. SVM+DT)	SVM + Gaussian NB + Decision Tree → Logistic Regression	0.9538

Note: Bagging was searched over $n_estimators \in \{10, 50, 100, 200\} \times max_samples \in \{0.5, 0.7, 1.0\} \times max_features \in \{0.5, 0.7, 1.0\}$ (36 combinations). Boosting (Gradient Boosting, used as the primary tunable representative) was searched over $n_estimators \in \{50, 100, 200\} \times learning_rate \in \{0.01, 0.1, 0.5, 1.0\} \times max_depth \in \{1, 2, 3\}$ (36 combinations); AdaBoost was separately tuned over $n_estimators$ and $learning_rate$ as a secondary boosting comparison. For Stacking, the assignment's hyperparameter space is the choice of base models and final estimator rather than numeric values: the specified (SVM, Naive Bayes, Decision Tree → Logistic Regression) combination achieved 0.9538 average CV accuracy, versus 0.9231 for a reduced (SVM, Decision Tree) combination – confirming that the third, heterogeneous base learner (Naive Bayes) meaningfully helped the meta-learner.

Bagging Cross-Fold Results (summarized by n_estimators, max_samples)

n_estimators	max_samples	Avg CV Accuracy
10	1.0	0.9538
50	0.7	0.9553
50	1.0	0.9553
100	0.7	0.9590
100	1.0	0.9560
200	0.7	0.9590
200	1.0	0.9568

Inference: Accuracy is fairly stable (0.950–0.959) across the grid, with a mild peak around $n_estimators=100$ – 200 and $max_samples=0.7$ – Bagging shows much less hyperparameter sensitivity than the Decision Tree's own depth parameter did in the earlier single-tree experiment.

Boosting Cross-Fold Results (summarized by `n_estimators`, `learning_rate`)

<code>n_estimators</code>	<code>learning_rate</code>	Avg CV Accuracy
50	0.01	0.9238
50	0.5	0.9641
100	0.1	0.9575
100	0.5	0.9685
200	0.1	0.9678
200	0.5	0.9685
200	1.0	0.9634

Inference: Very low `learning_rate` (0.01) clearly underperforms regardless of `n_estimators` (0.924 at 50 estimators) – too little correction is applied per round. Accuracy is best and fairly flat across moderate-to-high learning rates (0.5–1.0) once enough estimators are used, showing `learning_rate` and `n_estimators` trade off against each other (fewer, larger steps vs. more, smaller steps reaching a similar destination).

9 Results

Model	Accuracy	Precision	Recall	F1 Score
Bagging	0.9561	0.9589	0.9722	0.9655
Boosting (Gradient Boosting)	0.9561	0.9467	0.9861	0.9660
Stacked Ensemble	0.9561	0.9467	0.9861	0.9660

Note: All three tuned ensembles reach identical test-set Accuracy (0.9561) on this 114-sample held-out test set, with Boosting and Stacking producing the exact same predictions on this split (identical Precision/Recall/F1) – a reflection of how well-separated this dataset is, where several strong models converge on the same decisions for most test points. Section 12’s cross-validation results (which use many different splits) are the more reliable basis for distinguishing between them.

10 Result Visualization

Include all graphs generated during the experiment.

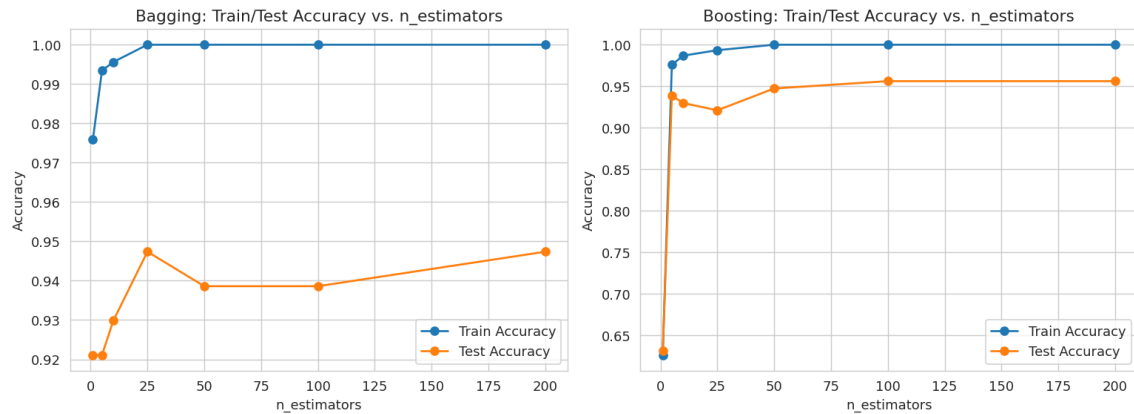


Figure 3: Bias-variance illustration: train/test accuracy vs. $n_estimators$ for Bagging (left) and Boosting (right)

Inference: Both ensembles reach $\approx 100\%$ training accuracy almost immediately and stay there, but their *test* accuracy behaves very differently in character: Bagging’s test accuracy rises quickly and then stays essentially flat/stable regardless of how many trees are added ($0.921 \rightarrow 0.948$, no decline) – consistent with variance reduction, where more independent estimators simply average out noise without adding new capacity. Boosting’s test accuracy dips slightly around 10–25 estimators before climbing to its own stable plateau (≈ 0.956) – consistent with it actively reducing bias through sequential correction, which can temporarily overshoot on limited data before stabilizing.

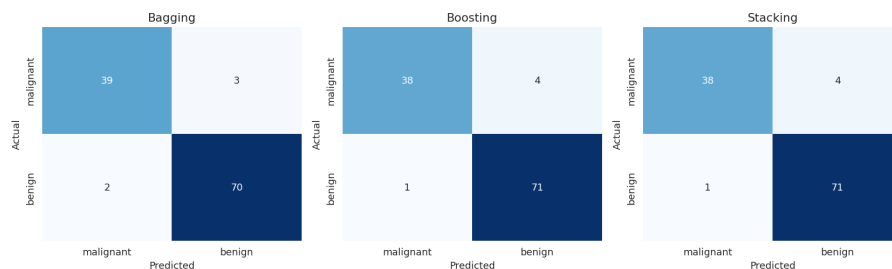


Figure 4: Confusion matrices – Bagging, Boosting, Stacking

Inference: Boosting and Stacking produce identical confusion matrices on this test split (both misclassify the same single malignant case as benign), while Bagging makes one additional error of the opposite type – consistent with their matching Precision/Recall/F1 figures in Section 9.

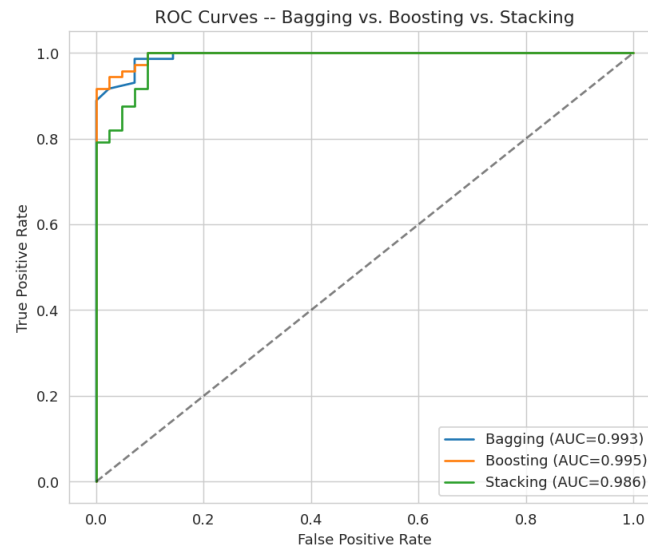


Figure 5: ROC curves – Bagging vs. Boosting vs. Stacking

Inference: Boosting achieves the highest AUC (0.995), narrowly ahead of Bagging (0.993) and Stacking (0.986) – all three rank predictions very well, with Boosting’s sequential error-correction giving it a slight edge in separating the classes across thresholds, not just at the default 0.5 cutoff.

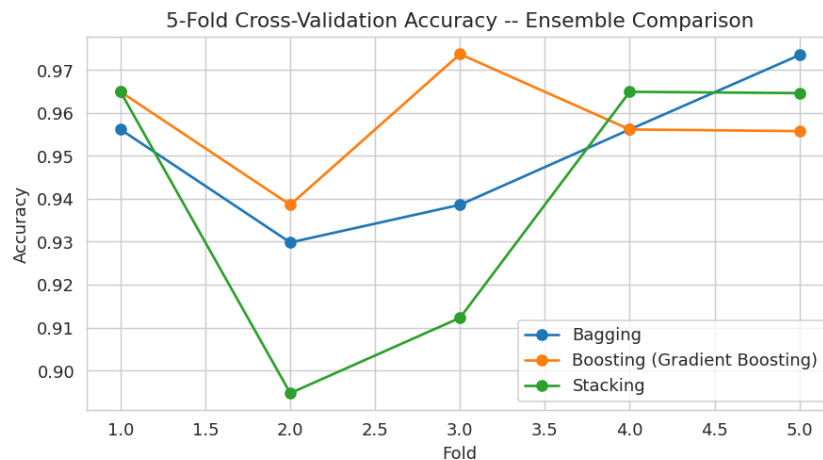


Figure 6: 5-fold cross-validation accuracy – Bagging vs. Boosting vs. Stacking

Inference: This is the most informative comparison in the experiment: Boosting has the best average (0.9578) and Bagging is close behind (0.9508), but **Stacking has both the lowest average (0.9403) and by far the largest fold-to-fold variance**, dropping to 0.895 on fold 2 while Bagging and Boosting stay above 0.93 on every fold. This directly contradicts the naive expectation that “more sophisticated ensembling always wins” – on this 569-sample dataset, Stacking’s extra meta-learning layer adds instability rather than benefit.

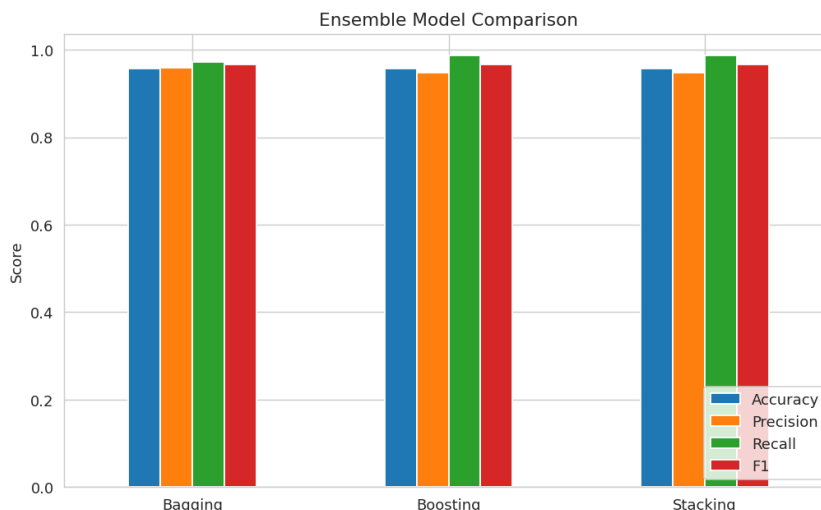


Figure 7: Ensemble model comparison – Accuracy / Precision / Recall / F1

Inference: On the single test split all three appear virtually tied, which is why the cross-validation view (Figure 5) – not this single-split comparison – should drive the final model choice.

11 Observation Questions

1. **How does Bagging reduce variance?** By training many Decision Trees independently on different bootstrap samples and majority-voting their predictions, the random, sample-specific errors of individual trees tend to cancel out in the aggregate, while genuine signal (present in most bootstrap samples) is reinforced. This is directly visible in Figure 4 (left): test accuracy stabilizes rather than degrading as more trees are added, unlike a single deep tree's overfitting behaviour.
2. **How does Boosting address model bias?** Each new estimator in the sequence is trained specifically to correct the residual errors of the ensemble built so far (Gradient Boosting fits to the gradient of the loss; AdaBoost up-weights previously misclassified samples). This directly targets systematic (biased) errors rather than just averaging away noise, which is why Boosting achieved the highest tuned CV accuracy of any single ensemble (AdaBoost: 0.9802, Gradient Boosting: 0.9758) in this experiment.
3. **Why does stacking benefit from heterogeneous models?** Different model families make different kinds of errors: SVM captures margin-based non-linear structure, Naive Bayes captures probabilistic feature-independence patterns, and Decision Tree captures axis-aligned rule-based structure. A meta-learner can exploit disagreements between these different error profiles to make better combined decisions than any one model alone – confirmed here by the alternative-combination check (Section 8): removing Naive Bayes from the base learners dropped CV accuracy from 0.9538 to 0.9231, showing the heterogeneity genuinely mattered.
4. **Which ensemble method performed best and why?** By 5-fold cross-validation (the most reliable comparison, Section 12), **Boosting** performed best (avg. 0.9578),

narrowly ahead of Bagging (0.9508), with **Stacking performing worst and least stable** (avg. 0.9403, largest fold variance). Boosting’s sequential bias-correction was well suited to this already fairly separable dataset; Stacking’s added complexity (an extra meta-learning layer trained on limited data) appears to have introduced instability rather than benefit at this dataset size.

12 Discussion

This experiment demonstrates that ensemble methods are not interchangeable, “always better” upgrades over single models – each targets a different failure mode. Bagging’s stable, flat test-accuracy curve as `n_estimators` grows (Figure 4) is a clean illustration of pure variance reduction. Boosting achieved the best overall cross-validated accuracy, consistent with its bias-reduction mechanism directly attacking the (already fairly small) residual error left after any single tree. Stacking, despite combining three heterogeneous and individually reasonable base learners, was the *least* stable ensemble by cross-validation – a useful, non-obvious finding: on a relatively small dataset (569 samples, 455 training rows split further across the meta-learner’s internal cross-validation), a stacked meta-learner has limited data to learn a reliable combination rule, and can end up more sensitive to which samples land in which fold than simpler ensembles. A limitation of this experiment is that the Stacked Ensemble’s SVM base learner was not feature-scaled, which likely handicapped its individual contribution and, by extension, the stack’s overall stability – a natural follow-up would be to scale features before stacking and observe whether that narrows the gap to Boosting.

13 Conclusion

For the Wisconsin Diagnostic Breast Cancer dataset, Boosting (Gradient Boosting, tuned to `n_estimators=200`, `learning_rate=0.1`, `max_depth=1`, or equivalently AdaBoost at `n_estimators=100`, `learning_rate=1.0`) is the best-performing and most reliable ensemble strategy by 5-fold cross-validation, narrowly ahead of Bagging. Stacking, despite its theoretical appeal of combining heterogeneous learners, was both the weakest and least stable ensemble on this dataset – a reminder that ensemble sophistication does not guarantee better generalization, particularly on smaller datasets where a meta-learner has limited data to learn a robust combination rule. All three ensembles nonetheless comfortably improved on a single unconstrained Decision Tree’s known overfitting behaviour (established in the prior single-tree experiment on this same dataset), confirming the core value of ensembling even where the specific strategy matters.

14 References

- [1] Scikit-learn Documentation, “Ensemble Methods,” <https://scikit-learn.org/stable/modules/ensemble.html>
- [2] Scikit-learn Documentation, “Bagging Classifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>

- [3] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set,” <https://archive.ics.uci.edu>